

FIȘĂ DE LUCRU NR. 2

Metode de Sortare a Elementelor unei Liste

Nume și prenume: _____

Clasa: _____

Data: _____



Obiective de învățare:

- ☐ Să descrii și să compari metodele de sortare: selecția minimului, lista de frecvențe și metoda bulelor
- ☐ Să trasezi execuția pas cu pas a fiecărui algoritm
- ☐ Să implementezi algoritmi de sortare în Python
- ☐ Să evaluezi eficiența fiecărei metode în funcție de contextul dat

PARTE TEORETICĂ

1. Sortarea prin Selecția Minimului

Idee de bază: Se caută succesiv minimul din subirul nesortat și se plasează la începutul acestuia.

Algoritm:

- ☐ Parcurge lista de la poziția 0 până la n-2
- ☐ La fiecare pas, găsește minimul din subșirul nesortat (de la poziția curentă la final)
- ☐ Interschimbă minimul găsit cu elementul de la poziția curentă

Complexitate: $O(n^2)$ — mereu același număr de comparații, indiferent de date

```
def selectie_minim(lst):  
    n = len(lst)  
    for i in range(n - 1):  
        idx_min = i  
        for j in range(i + 1, n):  
            if lst[j] < lst[idx_min]:  
                idx_min = j  
        # Interschimba  
        lst[i], lst[idx_min] = lst[idx_min], lst[i]  
    return lst  
  
# Exemplu  
note = [7, 4, 9, 3, 6, 8]  
print("Sortat:", selectie_minim(note))
```

2. Sortarea cu Lista de Frecvențe

Idee de bază: Contorizăm de câte ori apare fiecare valoare (dacă valorile sunt dintr-un interval limitat), apoi reconstituim lista sortată.

Condiție de aplicare: Valorile trebuie să fie întregi din un interval cunoscut și restrâns (de ex., note de la 1 la 10).

Complexitate: $O(n + k)$ unde k este dimensiunea intervalului — foarte eficientă pentru intervale mici!

```
def sortare_frecvente(lst, val_min, val_max):
    # Construiește lista de frecvențe
    frecvente = [0] * (val_max - val_min + 1)
    for x in lst:
        frecvente[x - val_min] += 1

    # Reconstruiește lista sortată
    sortat = []
    for i, freqv in enumerate(frecvente):
        sortat.extend([i + val_min] * freqv)
    return sortat

# Exemplu: note întregi de la 1 la 10
note = [7, 4, 9, 3, 6, 8, 4, 7, 10, 1]
print("Sortat:", sortare_frecvente(note, 1, 10))
```

3. Metoda Bulelor (Bubble Sort)

Idee de bază: Se compară perechi de elemente adiacente și se interschimbă cele aflate în ordine greșită. Elementele mari "se ridică" treptat spre capătul listei (ca bulele de aer în apă).

Observație importantă: Dacă la o parcurgere completă nu s-a efectuat nicio interschimbare, lista este deja sortată — algoritmul se poate opri mai devreme!

Complexitate: $O(n^2)$ în cazul defavorabil, $O(n)$ în cazul favorabil (lista deja sortată)

```
def metoda_bulelor(lst):
    n = len(lst)
    for i in range(n - 1):
        schimbat = False
        for j in range(n - 1 - i):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]
                schimbat = True
        if not schimbat:  # Lista deja sortată
            break
    return lst

# Exemplu
```

```
temperaturi = [22, 18, 25, 15, 30, 12]
print("Sortat:", metoda_bulelor(temperaturi))
```

4. Comparație metode de sortare

Criteriu	Selecția Minimului	Lista de Frecvențe	Metoda Bulelor
Complexitate	$O(n^2)$	$O(n + k)$	$O(n^2) / O(n)$
Restricții date	Nicio restricție	Întregi, interval limitat	Nicio restricție
Cel mai eficient când	Date puține	Note, vârste, coduri dintr-un interval fix	Date aproape sortate

EXERCIȚII

Exercițiul 1 — Trasarea execuției: Selecția Minimului (25 puncte)

Trasează pas cu pas execuția sortării prin selecția minimului pentru lista: [8, 3, 5, 1, 9, 2]

Pas	Lista curentă	Minim găsit	Pozițiile interschimbate	Lista după
0	[8, 3, 5, 1, 9, 2]			
1				
2				
3				
4				
5				

Exercițiul 2 — Alegerea metodei potrivite (20 puncte)

Pentru fiecare situație, alege metoda de sortare cea mai eficientă și justifică alegerea:

a) Trebuie să ordonezi 10.000 de note școlare (numere întregi de la 1 la 10).

Metodă: _____ Justificare: _____

b) Ordonezi 5 elevi după înălțime pentru o fotografie de grup.

Metodă: _____ Justificare: _____

c) Listezi produsele unui magazin online în ordine crescătoare a prețului (valori reale cu zecimale).

Metodă: _____ Justificare: _____

Exercițiul 3 — Implementare (30 puncte)

Cerință: Scrie un program Python complet care:

- ☐ Citește de la tastatură n note întregi (n introdus de utilizator)
- ☐ Sortează notele folosind metoda bulelor cu optimizare (oprire timpurie)
- ☐ Afișează lista sortată și media notelor (rotunjită la 2 zecimale)

Spațiu pentru rezolvare:

Notare orientativă:

- ☐ Exercițiul 1: 25 puncte
- ☐ Exercițiul 2: 20 puncte
- ☐ Exercițiul 3: 30 puncte
- ☐ 25 puncte activitate și participare

Total: 75 + 25 puncte = 100 puncte